

RAIN IN AUS ETL PIPELINE

DE241 DATA ARCHITECTURE
MAY 2026

PRESENTED BY
PARIYAKORN C. 66102010174

DATASET



JOE YOUNG AND 1 COLLABORATOR · UPDATED 5 YEARS AGO

▲

1879

<>

Code

⬇

Download

🟡

⋮

Rain in Australia

Predict next-day rain in Australia



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	
1	Date	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGustDir	WindGustSp	WindDir9am	WindDir3pm	WindSpeed9	WindSpeed3	Humidity9am	Humidity3pm	Pressure9am	Pressure3pm	Cloud9am	Cloud3pm	Temp9am	Temp3pm	RainToday	RainTomorrow	
2	1/12/2008	Albury	13.4	22.9	0.6	NA	NA	W	44	W	WNW	20	24	71	22	1007.7	1007.1	8	NA	16.9	21.8	No	No	
3	2/12/2008	Albury	7.4	25.1	0	NA	NA	WNW	44	NNW	WSW	4	22	44	25	1010.6	1007.8	NA	NA	17.2	24.3	No	No	
4	3/12/2008	Albury	12.9	25.7	0	NA	NA	WSW	46	W	WSW	19	26	38	30	1007.6	1008.7	NA	2	21	23.2	No	No	
5	4/12/2008	Albury	9.2	28	0	NA	NA	NE	24	SE	E	11	9	45	16	1017.6	1012.8	NA	NA	18.1	26.5	No	No	
6	5/12/2008	Albury	17.5	32.3	1	NA	NA	W	41	ENE	NW	7	20	82	33	1010.8	1006	7	8	17.8	29.7	No	No	
7	6/12/2008	Albury	14.6	29.7	0.2	NA	NA	WNW	56	W	W	19	24	55	23	1009.2	1005.4	NA	NA	20.6	28.9	No	No	
8	7/12/2008	Albury	14.3	25	0	NA	NA	W	50	SW	W	20	24	49	19	1009.6	1008.2	1	NA	18.1	24.6	No	No	
9	8/12/2008	Albury	7.7	26.7	0	NA	NA	W	35	SSE	W	6	17	48	19	1013.4	1010.1	NA	NA	16.3	25.5	No	No	
10	9/12/2008	Albury	9.7	31.9	0	NA	NA	NNW	80	SE	NW	7	28	42	9	1008.9	1003.6	NA	NA	18.3	30.2	No	Yes	

ETL PIPELINE

PART 1: LOAD (EXTRACT) จำลองการดึงข้อมูลรายวัน และจัดการ DATA MANAGEMENT

```
# -----  
# 1. LOAD (Extract): จำลองดึงข้อมูลรายวัน และจัดการ Data Management  
# -----  
def extract_data(ds, **kwargs):  
    print(f"Extracting data for date: {ds}")  
    df = pd.read_csv(RAW_FILE)  
  
    # Simulation: จำลองว่ามีข้อมูลเข้ามาแค่วันละ 1 วันตามที่ Airflow รัน (ds)  
    df_daily = df[df['Date'] == ds]  
  
    if df_daily.empty:  
        raise ValueError(f"No data found for {ds}")  
  
    # Data Management: ทำ Partitioning แยกโฟลเดอร์ตามปี/เดือน/วัน  
    dt = datetime.strptime(ds, '%Y-%m-%d')  
    partition_path = f"{STAGING_BASE}/year={dt.year}/month={dt.month:02d}/day={dt.day:02d}"  
    os.makedirs(partition_path, exist_ok=True)  
  
    out_path = f"{partition_path}/raw_weather.parquet"  
    df_daily.to_parquet(out_path, index=False)  
    print(f"Extracted {len(df_daily)} rows to {out_path}")
```

- ดึงและกรองข้อมูลรายวัน EXTRACT & SIMULATE
- ตรวจสอบความถูกต้อง DATA VALIDATION
- จัดระเบียบโครงสร้างโฟลเดอร์ DATA PARTITIONING
- บันทึกเป็นไฟล์ SAVE AS PARQUET

ETL PIPELINE

PART 2: TRANSFORM แปลงและทำความสะอาดข้อมูล

- ค้นหาและโหลดข้อมูลดิบ LOAD RAW DATA
- ตัดคอลัมน์ที่ไม่จำเป็น DATA PRUNING
- จัดการค่าว่างแบบมีกลยุทธ์ NULL HANDLING & IMPUTATION
- ตรวจสอบคุณภาพและปรับแต่ง SANITY CHECK & FORMATTING
- บันทึกผลลัพธ์ SAVE CLEANED DATA

```
35 # 2. TRANSFORM: ทำ Data Quality Check และจัดการค่าว่าง
36 # -----
37 def transform_data(ds, **kwargs):
38     dt = datetime.strptime(ds, '%Y-%m-%d')
39     partition_path = f"{STAGING_BASE}/year={dt.year}/month={dt.month:02d}/day={dt.day:02d}"
40     in_path = f"{partition_path}/raw_weather.parquet"
41
42     df = pd.read_parquet(in_path)
43
44     # วิธีแก้ที่ 1: Data Pruning (ตัดคอลัมน์ที่ไม่ใช่ข้อมูล)
45     columns_to_keep = ['Date', 'Location', 'MinTemp', 'MaxTemp', 'Rainfall', 'Humidity9am', 'Humidity3pm']
46     df = df[columns_to_keep].copy()
47
48     # Data Quality 1: ลบแถวที่มีค่าว่างในคอลัมน์สำคัญ
49     df = df.dropna(subset=['MinTemp', 'MaxTemp', 'Location'])
50
51     # วิธีแก้ที่ 2: Data Imputation (เติมค่าว่างด้วยค่าเฉลี่ย)
52     df['Humidity9am'] = df['Humidity9am'].fillna(df['Humidity9am'].mean())
53     df['Humidity3pm'] = df['Humidity3pm'].fillna(df['Humidity3pm'].mean())
54
55     # ✨ สิ่งที่เราเพิ่มเข้ามา: ปัดเศษทศนิยมทุกคอลัมน์ให้เป็นตัวเลขให้เหลือ 1 ตำแหน่ง
56     df = df.round(1)
57
58     # Data Quality 2: ตรวจสอบความสมเหตุสมผลของข้อมูล (Sanity Check)
59     df = df[df['MaxTemp'] >= df['MinTemp']]
60     df['Rainfall'] = df['Rainfall'].fillna(0)
61     df = df[df['Rainfall'] >= 0]
62
63     # แปลง Location เป็นตัวพิมพ์เล็ก
64     df['Location'] = df['Location'].str.lower()
65
66     # เซฟไฟล์เพิ่มเติม ทั้ง Parquet และ CSV
67     out_parquet = f"{partition_path}/clean_weather.parquet"
68     df.to_parquet(out_parquet, index=False)
69
70     out_csv = f"{partition_path}/clean_weather.csv"
71     df.to_csv(out_csv, index=False)
72
73     print(f"Transformed data. Remaining rows: {len(df)}")
```

	A	B	C	D	E	F	G
1	Date	Location	MinTemp	MaxTemp	Rainfall	Humidity9am	Humidity3pm
2	1/12/2008	albury	13.4	22.9	0.6	71	22
3	1/12/2008	penrith	15.2	32.6	0	35	23
4	1/12/2008	sydney	17.6	31.3	0	29	21
5	1/12/2008	canberra	13.6	25.2	0	31	28
6	1/12/2008	tuggeranong	13.1	24.2	0	36	34
7	1/12/2008	mountginini	5.2	13	0.4	52.9	41.2
8	1/12/2008	ballarat	5.8	17.6	3.2	71	37
9	1/12/2008	bendigo	9.1	21.7	0	47	23
10	1/12/2008	melbourne	11.8	20.8	1.4	52	32

ETL PIPELINE

PART 3: LOAD การนำเข้าฐานข้อมูล

- ค้นหาและโหลดข้อมูลที่สะอาดแล้ว LOAD CLEAN DATA
- เชื่อมต่อฐานข้อมูลอย่างปลอดภัย SECURE DB CONNECTION
- หัวใจสำคัญ: ระบบลบข้อมูลเก่า IDEMPOTENCY LOGIC
- ดักจับข้อผิดพลาดกรณีรันครั้งแรกสุด ERROR HANDLING

```
75 # -----
76 # 3. INGEST (Load): นำเข้า Database แบบ Star Schema และ Security
77 # -----
78 def ingest_data(ds, **kwargs):
79     dt = datetime.strptime(ds, '%Y-%m-%d')
80     partition_path = f"{STAGING_BASE}/year={dt.year}/month={dt.month:02d}/day={dt.day:02d}"
81     in_path = f"{partition_path}/clean_weather.parquet"
82
83     df = pd.read_parquet(in_path)
84
85     pg_hook = PostgresHook(postgres_conn_id='postgres_weather_db')
86     engine = pg_hook.get_sqlalchemy_engine()
87
88     # =====
89     # ✨ IDEMPOTENCY LOGIC: ลบข้อมูลของวันนั้นทิ้งก่อน (Delete Before Insert)
90     # =====
91     with engine.begin() as conn:
92         try:
93             # สั่งลบแถวที่มีวันที่ตรงกับตัวแปร ds
94             conn.execute(text(f"DELETE FROM fact_weather WHERE 'Date' = '{ds}';"))
95             conn.execute(text(f"DELETE FROM dim_date WHERE full_date = '{ds}';"))
96             # สำหรับสถานที่ (Location) เราลบทิ้งทั้งหมดก่อนชั่วคราวเพื่อป้องกันรายชื่อเมืองเบิ้ล
97             conn.execute(text("DELETE FROM dim_location;"))
98         except Exception:
99             # หากรันครั้งแรกสุด ตารางอาจจะยังไม่ถูกสร้าง มันจะเกิด Error ลบไม่ได้ ให้เราข้ามไปเลย
100             pass
101     # =====
```


ETL PIPELINE

PART 4: แยกชิ้นส่วนข้อมูลและนำไปจัดเรียงลง DATABASE

- จัดเรียงตารางสถานที่ (INSERT DIM_LOCATION)
- จัดเรียงตารางมิติเวลา (INSERT DIM_DATE)
- จัดเรียงตารางข้อเท็จจริง (INSERT FACT_WEATHER)
- พิมพ์รายงานผล (LOGGING)

```
103 # หลังจากเคลียร์พื้นที่เสร็จแล้ว ก็ทำการ Insert ข้อมูลใหม่ลงไปตามปกติ
104 dim_location = df[['Location']].drop_duplicates().rename(columns={'Location': 'location_name'})
105 dim_location.to_sql('dim_location', engine, if_exists='append', index=False)
106
107 dim_date = pd.DataFrame([
108     'full_date': ds,
109     'year': dt.year,
110     'month': dt.month,
111     'day': dt.day
112 ])
113 dim_date.to_sql('dim_date', engine, if_exists='append', index=False)
114
115 fact_weather = df[['Date', 'Location', 'MinTemp', 'MaxTemp', 'Rainfall', 'Humidity9am', 'Humidity3pm']].copy()
116 fact_weather.to_sql('fact_weather', engine, if_exists='append', index=False)
117
118 print(f"Ingested {len(fact_weather)} rows into Star Schema with Idempotency.")
```

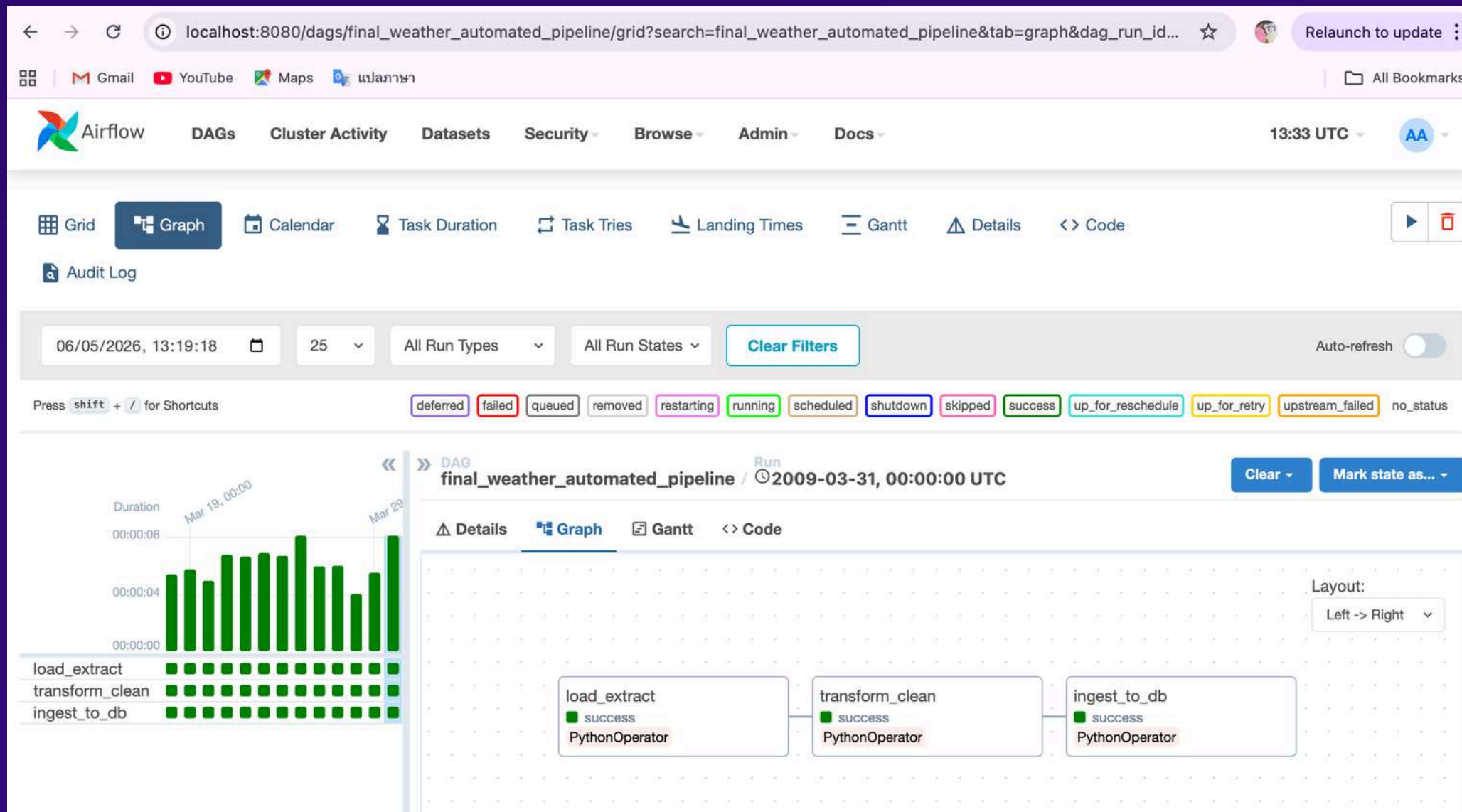

ETL PIPELINE

PART 5: ORCHESTRATOR

```
119 # -----
120 # ตั้งค่า Airflow DAG
121 # -----
122 default_args = {
123     'owner': 'data_engineer',
124     'retries': 1,
125     'retry_delay': timedelta(minutes=2),
126 }
127
128 with DAG(
129     'weather_pipeline_final',
130     default_args=default_args,
131     schedule_interval='@daily',      # รันอัตโนมัติทุกวัน (Automated)
132     start_date=datetime(2008, 12, 1), # วันแรกของ Dataset
133     end_date=datetime(2009, 3, 31),   # <-- วันสิ้นสุด (มีนาคม 2009)
134     catchup=True,                    # สั่งให้รันย้อนหลังเพื่อจำลองการทำงานรายวัน
135     max_active_runs=1,               # บังคับให้รันทีละ 1 วันเท่านั้น
136     tags=['weather', 'etl', 'star_schema'],
137 ) as dag:
138
139     t1 = PythonOperator(task_id='load_extract', python_callable=extract_data)
140     t2 = PythonOperator(task_id='transform_clean', python_callable=transform_data)
141     t3 = PythonOperator(task_id='ingest_to_db', python_callable=ingest_data)
142
143     t1 >> t2 >> t3
```

- ตั้งกฎความปลอดภัยพื้นฐาน (DEFAULT_ARGS)
- ตั้งค่าโปรไฟล์และตารางเวลาของ DAG (WITH DAG(...))
- ผูกฟังก์ชันเข้ากับกล่องงาน (PYTHONOPERATOR)
- จัดลำดับการทำงาน (T1 >> T2 >> T3)

AIRFLOW



The screenshot displays the 'Add Connection' form in the Apache Airflow web interface. The form is used to create a new connection for the DAG. The fields are as follows:

- Connection Id: postgres_weather_db
- Connection Type: Postgres
- Description: (empty)
- Host: postgres
- Schema: (empty)
- Login: airflow
- Password: (masked with dots)
- Port: 5432

The form also includes a 'Relaunch to update' button and a 'Clear' button.

DIRECTORY STRUCTURE

1 INFRASTRUCTURE SETUP

- DOCKER-COMPOSE.YAML
- .ENV เก็บรหัสผ่านฐานข้อมูล (MY_SECRET_POSTGRES_PASSWORD) และกุญแจเข้ารหัส (FERNET_KEY)

2 RAW DATA

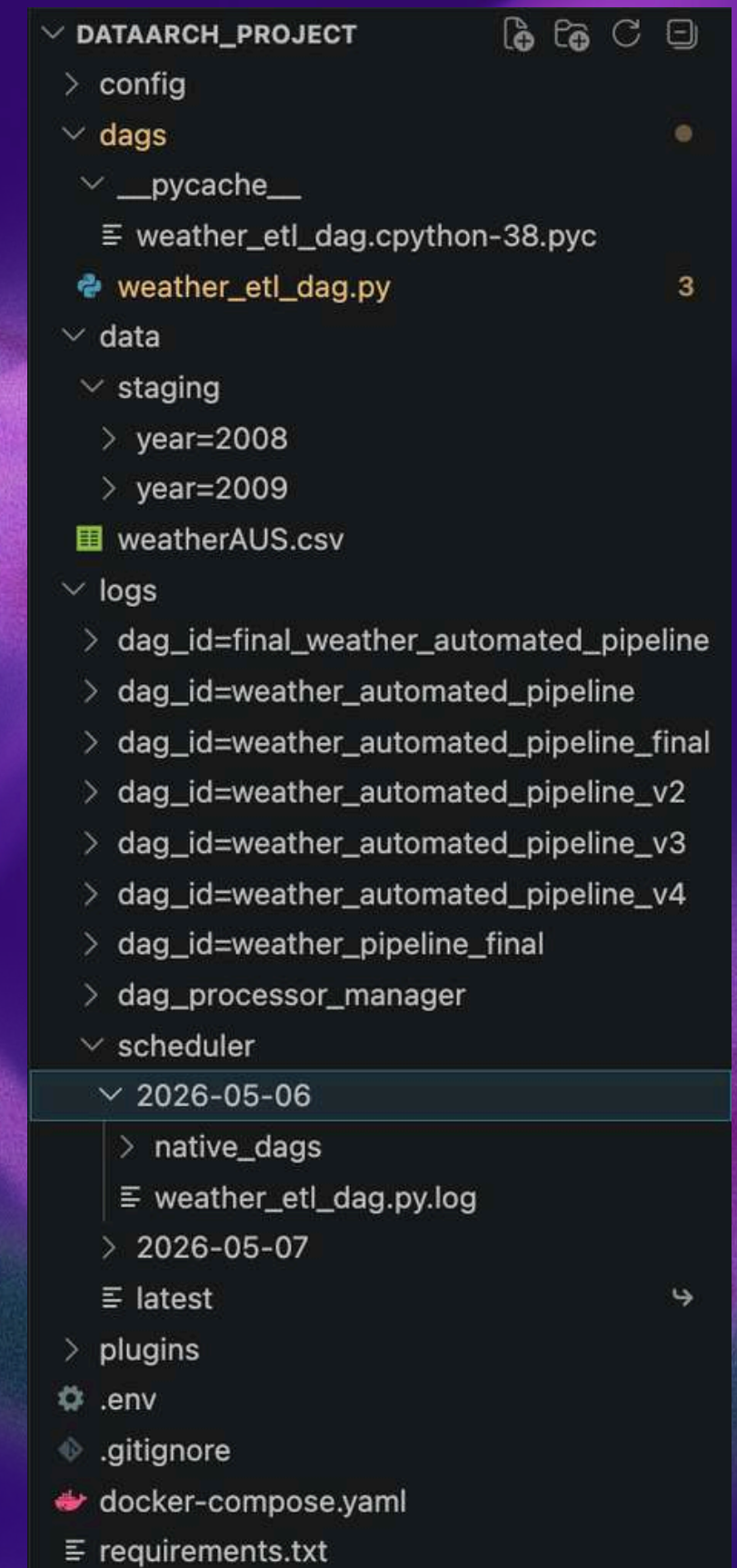
- ไฟล์ข้อมูล DATA/WEATHERAUS.CSV

3 THE PIPELINE LOGIC

- DAGS/WEATHER_ETL_DAG.PY
- DAGS/__PYCACHE__/

4 EXECUTION OUTPUTS

- LOGS/ สุ่มบันทึกการทำงานของ AIRFLOW
- DATA/STAGING (YEAR=.../MONTH=.../DAY=...) ภายใน DAY=01 ประกอบด้วย RAW_WEATHER.PARQUET, CLEAN_WEATHER.PARQUET และ CLEAN_WEATHER.CSV



SECURITY & BEST PRACTICES

INFRASTRUCTURE SECURITY:

- ใช้ไฟล์ .ENV ซ่อนรหัสผ่านของฐานข้อมูล
- ใช้ไฟล์ .GITIGNORE เพื่อป้องกันไม่ให้ดาต้าลับหลุดรอดออกไปบน GITHUB

CREDENTIAL MANAGEMENT:

- ตั้งค่า FERNET KEY ใน AIRFLOW เพื่อเข้ารหัส (ENCRYPT) รหัสผ่านทั้งหมดที่อยู่ในระบบ ป้องกันการถูกแฮกข้อมูลใน DATABASE

.ENV

```
🔧 .env
1  # Database Secrets
2  AIRFLOW_UID=50000
3  MY_SECRET_POSTGRES_PASSWORD=airflow
4  AIRFLOW__CORE__FERNET_KEY=J0p9_A1b2C3d4E5f6G7h8I9j0K1l2M3n4O5p6Q7r8S9=
```

.GITIGNORE

```
🔍 .gitignore
1  .env
2  __pycache__/
3  logs/
4  data/
```

WEATHER_ETL_DAG.PY

```
85  pg_hook = PostgresHook(postgres_conn_id='postgres_weather_db')
```


DATA MODELING

ออกแบบโครงสร้างแบบ STAR SCHEMA

DIMENSION TABLES:

- DIM_LOCATION (เก็บรายชื่อเมือง)
- DIM_DATE (เก็บมิติของเวลา) เพื่อลดความซ้ำซ้อนของข้อมูล

FACT TABLE:

- FACT_WEATHER เก็บเฉพาะตัวเลขที่ใช้วัดผล (MEASURES) เช่น อุณหภูมิ ปริมาณฝน และเก็บ FOREIGN KEY ที่เชื่อมไปยังตาราง DIMENSION

ช่วยให้ DATA ANALYST สามารถนำไปเขียนคำสั่ง JOIN เพื่อหา INSIGHT หรือทำ DASHBOARD ได้อย่างรวดเร็วและมีประสิทธิภาพ

PROBLEMS 3	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
List of relations				
Schema	Name	Type	Owner	
public	ab_permission	table	airflow	
public	ab_permission_view	table	airflow	
public	ab_permission_view_role	table	airflow	
public	ab_register_user	table	airflow	
public	ab_role	table	airflow	
public	ab_user	table	airflow	
public	ab_user_role	table	airflow	
public	ab_view_menu	table	airflow	
public	alembic_version	table	airflow	
public	callback_request	table	airflow	
public	celery_taskmeta	table	airflow	
public	celery_tasksetmeta	table	airflow	
public	connection	table	airflow	
public	dag	table	airflow	
public	dag_code	table	airflow	
public	dag_owner_attributes	table	airflow	
public	dag_pickle	table	airflow	
public	dag_run	table	airflow	
public	dag_run_note	table	airflow	
public	dag_schedule_dataset_reference	table	airflow	
public	dag_tag	table	airflow	
public	dag_warning	table	airflow	
public	dagrun_dataset_event	table	airflow	
public	dataset	table	airflow	
public	dataset_dag_run_queue	table	airflow	
public	dataset_event	table	airflow	
public	dim_date	table	airflow	
public	dim_location	table	airflow	
public	fact_weather	table	airflow	

DATA QUERY

```
"
```

location_name	total_rainfall_mm	rainy_days_count
cairns	1835.2	70
townsville	1812.2	69
darwin	1348.2	85
coffsharbour	659.8	45
goldcoast	475.8	59

```
(5 rows)
```

5 อันดับเมืองที่ฝนตกมากที่สุด

```
"
```

location_name	avg_temp_swing	max_single_day_swing
mildura	16.5	23.5
pearceraaf	16.4	26.5
nuriootpa	16.3	25.7
bendigo	16.3	26.6
waggawagga	16.2	26.6

```
(5 rows)
```

เมืองที่มีอากาศแปรปรวน

location_name	days_recorded	avg_max_temp
alicesprings	121	35.6
woomera	90	34.5
cobar	90	33.7
moree	90	32.7
pearceraaf	82	32.7
waggawagga	90	32.6
mildura	90	32.5
darwin	121	32.1
perthairport	90	31.8
cairns	121	31.5

```
(10 rows)
```

10 อันดับเมืองที่ร้อนที่สุด